Regno: 26MAG0064
Name: AADITYA SOGANI

"""
Lab Assignment: Spectral Clustering with Different Similarity Kernels
Dataset: CIC-IDS2017 - DoS-Wednesday (stratified subsample, 1511 flows, 6 classes) """

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder from
sklearn.decomposition import PCA
from sklearn.cluster import SpectralClustering
from sklearn.metrics.pairwise import rbf_kernel, polynomial_kernel, cosine_similarity from
sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score, silhouette_score

# ---------------------------------
# 1) Load and preprocess the dataset #
# ---------------------------------
df = pd.read_parquet("spectral_sample.parquet")
print("Sample shape:", df.shape)
print(df["Label"].value_counts())

y_true_labels = df["Label"] le
= LabelEncoder()
y_true = le.fit_transform(y_true_labels) n_clusters =
len(le.classes_)
print(f"\nNumber of true classes: {n_clusters}")
"""output:
Sample shape: (1511, 78)
Label
Benign                  300
DoS GoldenEye           300
DoS Hulk                 300
DoS Slowhttptest        300
DoS slowloris             300
Heartbleed                 11
Name: count, dtype:   int64

Number of true classes: 6 """

X = df.drop(columns=["Label"]).select_dtypes(include=[np.number]).copy()
X = X.replace([np.inf, -np.inf], np.nan).dropna() y_true
= y_true[X.index]
y_true_labels = y_true_labels.loc[X.index]

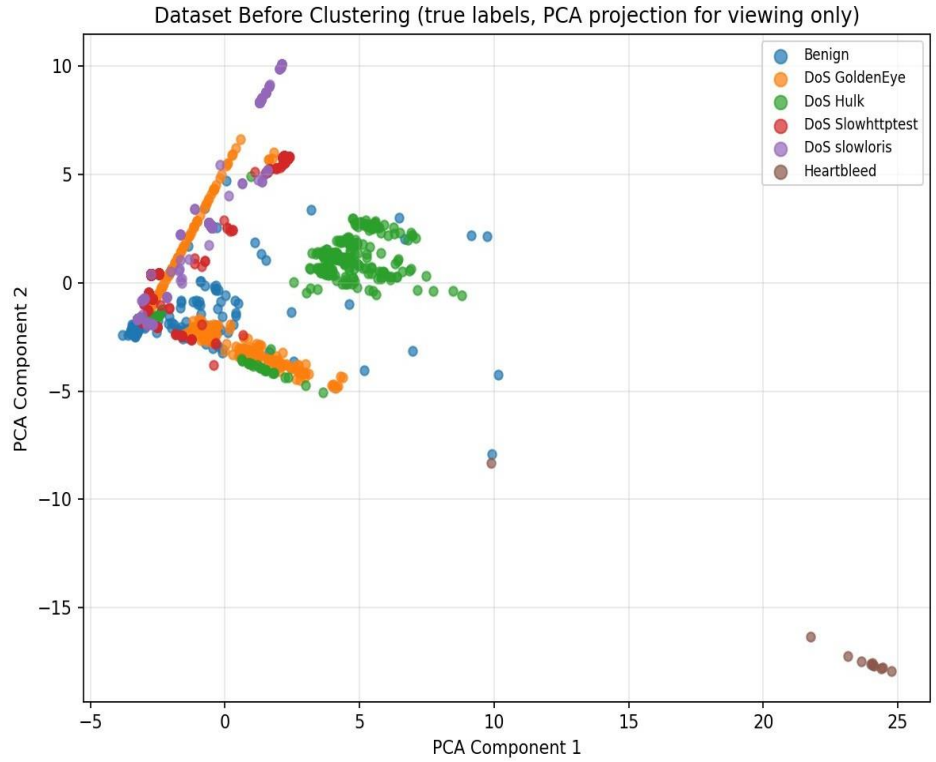
# Standardize -- required before computing any similarity/distance-based kernel, # for the
same reason as in PCA: features must be on comparable scales.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ---------------------------------
# 2) Visualize the dataset before clustering #
# ---------------------------------
```

77 dimensions can't be plotted directly -- project to 2D with PCA purely to *see* # the data.
This is only for visualization; clustering itself runs on the full
standardized feature space, not on these 2 PCA components. pca_vis =
PCA(n_components=2)

```
X_vis = pca_vis.fit_transform(X_scaled)

plt.figure(figsize=(8, 6)) for
label in le.classes_:
    mask = (y_true_labels == label).values
    plt.scatter(X_vis[mask, 0], X_vis[mask, 1], s=25, alpha=0.7, label=label) plt.xlabel("PCA
Component 1")
plt.ylabel("PCA Component 2")
plt.title("Dataset Before Clustering (true labels, PCA projection for viewing only)")
plt.legend(markerscale=1.5, fontsize=8)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("01_dataset_before_clustering.png", dpi=150) plt.show()
```




```
# ----------------------------------
# 3) Construct similarity matrices with different kernels #
# ----------------------------------
# RBF (Gaussian) kernel: similarity decays smoothly with Euclidean distance. gamma =
1.0 / X_scaled.shape[1]
S_rbf = rbf_kernel(X_scaled, gamma=gamma)

# Polynomial kernel: similarity based on (a . b + c)^degree S_poly =
polynomial_kernel(X_scaled, degree=3, coef0=1)
S_poly = (S_poly - S_poly.min()) / (S_poly.max() - S_poly.min())

# Cosine kernel: similarity based on the angle between vectors S_cosine
= cosine_similarity(X_scaled)
```

S_cosine = (S_cosine + 1) / 2

kernels = {

```
    "RBF (Gaussian)": S_rbf,
    "Polynomial": S_poly,
    "Cosine": S_cosine,
}

# ------------------------------
# 4) Perform Spectral Clustering with each similarity matrix #
# ------------------------------
results = {} cluster_labels
= {}

for name, S in kernels.items(): sc =
    SpectralClustering(
        n_clusters=n_clusters,
        affinity="precomputed",
        assign_labels="kmeans",
        random_state=42,
    )
    pred = sc.fit_predict(S)
    cluster_labels[name] = pred

    ari = adjusted_rand_score(y_true, pred)
    nmi = normalized_mutual_info_score(y_true, pred) sil =
    silhouette_score(X_scaled, pred)

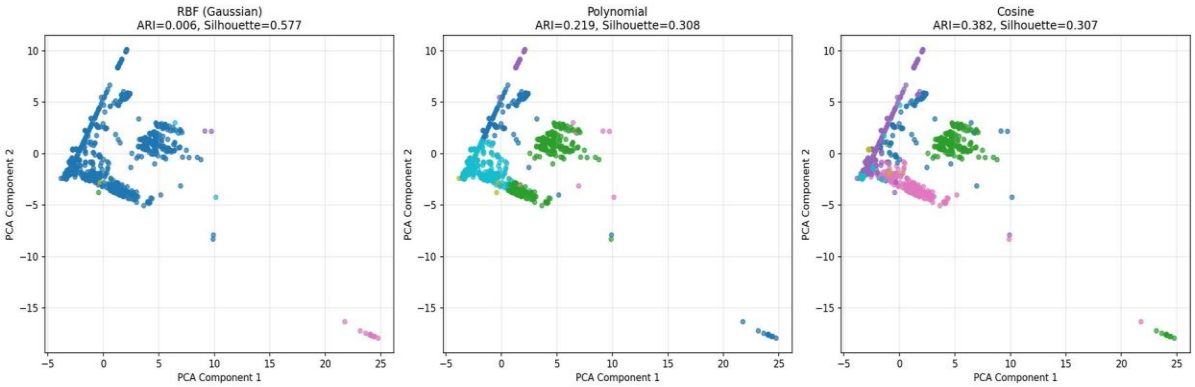
    results[name] = {"ARI": ari, "NMI": nmi, "Silhouette": sil} print(f"{name}:
    ARI={ari:.4f}, NMI={nmi:.4f}, Silhouette={sil:.4f}")

results_df = pd.DataFrame(results).T
results_df.to_csv("clustering_metrics.csv") print("\n",
results_df)
"""output:
RBF (Gaussian): ARI=0.0065, NMI=0.0547, Silhouette=0.5766
Polynomial: ARI=0.2190, NMI=0.3521, Silhouette=0.3078
Cosine: ARI=0.3822, NMI=0.5041, Silhouette=0.3065

                          ARI          NMI  Silhouette
RBF (Gaussian)  0.006491  0.054725        0.576590
Polynomial          0.218994  0.352070      0.307839
Cosine              0.382212  0.504135
                    0.306531 """

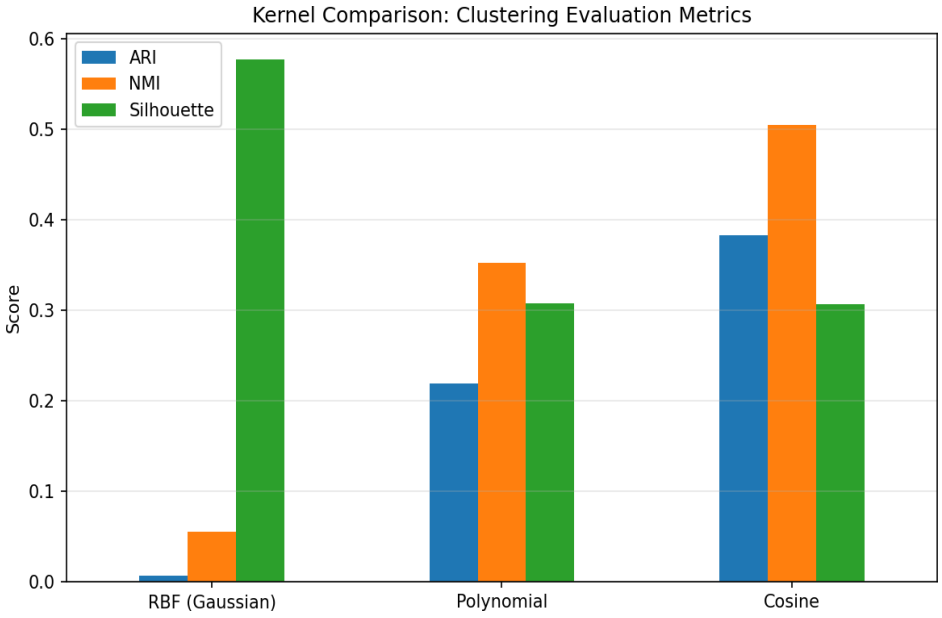
# ------------------------------
# 5) Visualize clustering results (same PCA projection, colored by predicted cluster) #
# ------------------------------
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
for ax, (name, pred) in zip(axes, cluster_labels.items()):
    ax.scatter(X_vis[:, 0], X_vis[:, 1], c=pred, cmap="tab10", s=20, alpha=0.7)
    ax.set_title(f"{name}\nARI={results[name]['ARI']:.3f}, Silhouette={results[name]
['Silhouette']:.3f}") ax.set_xlabel("PCA
    Component 1")
```

```
        ax.set_ylabel("PCA Component 2")
        ax.grid(True, alpha=0.3)
plt.tight_layout() plt.savefig("02_clustering_results_by_kernel.png",
dpi=150) plt.show()
```




```
# ----------------------------------
# 6) Compare kernel performance #
# ----------------------------------
plt.figure(figsize=(8, 5))
results_df[["ARI", "NMI", "Silhouette"]].plot(kind="bar", ax=plt.gca())
plt.title("Kernel Comparison: Clustering Evaluation Metrics") plt.ylabel("Score")
plt.xticks(rotation=0) plt.grid(True,
alpha=0.3, axis="y") plt.tight_layout()
plt.savefig("03_kernel_comparison_bar.png", dpi=150)
plt.show()

print("\nSaved: 01_dataset_before_clustering.png, 02_clustering_results_by_kernel.png,")
print("               03_kernel_comparison_bar.png, clustering_metrics.csv")
```




```
"""output:
Saved: 01_dataset_before_clustering.png, 02_clustering_results_by_kernel.png,
        03_kernel_comparison_bar.png, clustering_metrics.csv
"""
```